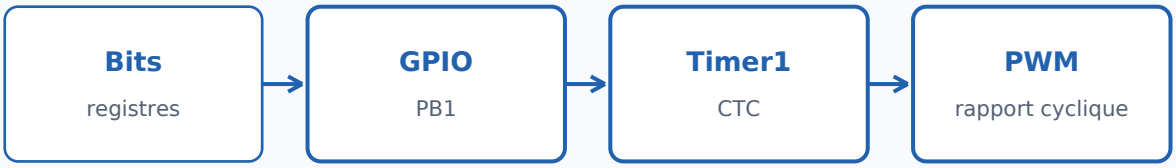
COURS PRATIQUE • EX00 À EX03

ATmega328P Comprendre avant de coder

GPIO, Timer1 et PWM — guide illustré en français

Ce document rassemble les explications de notre échange et les organise en un parcours de révision. Les schémas montrent ce qui se passe dans le matériel ; les exemples montrent comment traduire ce comportement en C.



Du changement d’état logiciel au signal produit par le matériel

Hypothèses des calculs
Horloge effective : 16 MHz. Timer1 : prescaler 256. LED de l’exercice : supposée reliée à PB1 / OC1A et active à HIGH. Le schéma de ta carte doit confirmer le câblage et les broches SW1 / SW2.

Les règles des exercices reprises ici proviennent de notre échange. Si ton sujet précise une autre horloge ou d’autres contraintes, adapte les exemples à ce sujet.

Références : ton guide visuel fourni, la datasheet 7810D–AVR–01/15 et la clarification Microchip sur le Fast PWM. Les liens et repères sont regroupés à la fin.

PLAN DE LECTURE

Ton parcours

Pages de ce document	À comprendre
3–4	Broches, registres, bits et masques
5–7	DDRx, PORTx, PINx et ex00
8–10	Horloge, compteur, prescaler et registres Timer1
11–13	Ex01 : CTC et bascule matérielle à 1 Hz
14–17	Ex02 : PWM à 1 Hz, rapport cyclique 10 %
18–21	Ex03 : boutons, rebonds et rapport cyclique variable
22–24	Méthode, fiches de révision et références

Les quatre exercices en une vue

Exercice	Commande LED	Temporisation	Boucle principale
00	PORTB	Attente logicielle	Bascule + attente
01	OC1A	Timer1 / CTC	Vide
02	OC1A	Timer1 / Fast PWM	Vide
03	OC1A	Timer1 / Fast PWM	Lecture des boutons

Lis d’abord les bases GPIO, puis ex01. Ex02 reprend le même timer en changeant son mode. Ex03 conserve la PWM et ajoute seulement une nouvelle valeur de comparaison pilotée par les boutons.

BASES • 1

Broches, ports et registres

PB1 désigne la broche logique numéro 1 du port B. Le nom ne correspond pas au numéro de patte du boîtier : celui-ci dépend du boîtier utilisé. Les noms D2, SW1 et SW2 appartiennent à la carte électronique.

Un registre commande une fonction du matériel

Un registre est une petite mémoire accessible au programme. Certains registres contiennent une valeur ; d'autres réunissent des bits de configuration. Avec **<avr/io.h>**, les noms de registres et de bits sont fournis par les en-têtes AVR.

Nom	Rôle
DDRB	Choisit entrée ou sortie pour les broches du port B
PORTB	Commande la sortie GPIO ou la pull-up en entrée
PINB	Lit les niveaux des broches du port B
TCCR1A / TCCR1B	Configurent Timer1
TCNT1	Contient la valeur du compteur
OCR1A / ICR1	Valeurs utilisées selon le mode du timer

Deux niveaux à distinguer
La datasheet explique PB1 et OC1A. Le schéma de la carte explique quelle LED ou quel bouton est connecté à cette broche.

Où vérifier : Guide fourni p. 5–6 et 13 ; datasheet §1 et §13.3.

BASES • 2

Bits et opérations bitwise

Un registre GPIO contient 8 bits, numérotés de 0 à 7. Un masque sert à sélectionner un bit sans modifier les autres. **1 << PB1** décale la valeur 1 jusqu'à la position de PB1.

Masque pour PB1



1 << PB1 = 00000010 en binaire = 2 en décimal

Intention	Expression C	Effet
Mettre PB1 à 1	registre |= (1 << PB1);	OR ajoute le bit sélectionné
Mettre PB1 à 0	registre &= ~(1 << PB1);	AND avec le masque inversé
Inverser PB1	registre ^= (1 << PB1);	XOR : 0 devient 1, 1 devient 0
Tester PB1	registre & (1 << PB1)	Résultat nul ou non nul

= et |= ne font pas la même chose

= remplace toute la valeur du registre. **|=** conserve les bits déjà présents et ajoute ceux du masque. Pour configurer un timer depuis un état connu, une affectation complète rend aussi explicites les bits à zéro.

```
DDRB |= (1 << PB1);  // conserve les autres directions
PORTB ^= (1 << PB1); // inverse uniquement PB1
```

Où vérifier : Guide fourni p. 7 ; datasheet §13.1 et §13.4.

GPIO • DIRECTION ET COMMANDE

DDRx et PORTx

DDRx : choisir le sens de la broche

Pour le bit correspondant à une broche : **0 = entrée**, **1 = sortie**. Pour commander la LED sur PB1, il faut donc configurer PB1 en sortie.

```
DDRB |= (1 << PB1);
```

PORTx : sa signification dépend de DDRx

DDR	PORT	Configuration GPIO
0	0	Entrée sans pull-up interne
0	1	Entrée avec pull-up interne, si autorisée globalement
1	0	Sortie au niveau LOW
1	1	Sortie au niveau HIGH

En sortie GPIO, écrire dans PORTB commande le niveau de la broche. En entrée, le même bit sert à activer la résistance de rappel interne. Une fonction alternative comme OC1A peut prendre la commande de la sortie lorsque le périphérique est configuré.

```
PORTB |= (1 << PB1);   // HIGH en sortie GPIO
PORTB &= ~(1 << PB1);  // LOW en sortie GPIO
```

DDRB reste nécessaire pour OC1A : le périphérique produit le signal, mais la broche doit être configurée en sortie pour le rendre visible à l’extérieur.

Où vérifier : Guide fourni p. 8–9 et 13 ; datasheet §13.2.1 et §13.3.

GPIO • LECTURE ET CIRCUIT

PINx et la LED

PINx permet de lire l'état logique présent sur les broches. **PORTx** représente la commande de sortie ou la configuration de la pull-up ; ce sont des rôles différents.

```
if (PINB & (1 << PB1))
{
    // niveau lu : HIGH
}
```

Exemple de LED active à HIGH

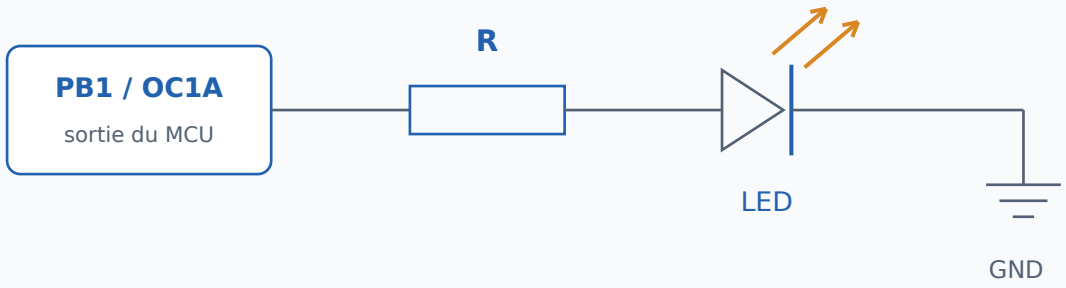


Schéma de principe : résistance et polarité à vérifier sur ta carte.

Dans ce schéma, HIGH fait circuler du courant dans la LED et LOW l'éteint. Une LED reliée à VCC peut au contraire être active à LOW. La résistance limite le courant : sa valeur dépend du montage et des limites électriques à vérifier.

La particularité AVR de PINB

Écrire un **1** dans un bit de PINB inverse le bit correspondant de PORTB. Ce n'est pas une écriture de niveau logique classique.

```
PINB = (1 << PB1); // bascule le latch PORTB de PB1
```

Pour l'ex00, **PORTB ^= (1 << PB1)** exprime plus directement l'opération XOR demandée. Évite une écriture de type PINB |= masque : les bits lus à 1 peuvent aussi être réécrits à 1 et provoquer d'autres bascules.

Où vérifier : Guide fourni p. 10–11 ; datasheet §13.2.2, §13.2.4 et §28.

EX00 • ATTENDRE AVEC LE CPU

Faire basculer la LED en logiciel

L’ex00 sert à comprendre les GPIO et les opérations bitwise. La LED est commandée par PORTB. Le programme change son état, attend avec une boucle logicielle, puis recommence.



La bascule et l’attente sont répétées dans while (1).

```
#include <avr/io.h>

int main(void)
{
    DDRB |= (1 << PB1);

    while (1)
    {
        PORTB ^= (1 << PB1);
        // Attente logicielle à calibrer selon le sujet.
    }
}
```

Une bascule n’est pas une période complète

Pour un clignotement à 1 Hz avec 50 % de temps allumé, il faut une bascule toutes les 0,5 s. Deux bascules donnent une période de 1 s.

> Une boucle C n’a pas une durée universelle : le compilateur et ses optimisations changent le nombre d’instructions. Une boucle sans effet observable peut être supprimée. Une variable volatile peut empêcher sa suppression, mais ne garantit pas une durée précise.

Si le sujet interdit if/else pour inverser l’état, XOR suffit. Si les délais de bibliothèque sont interdits dans cet exercice, ne remplace pas la boucle par _delay_ms().

Où vérifier : Guide fourni p. 7–11 et 14 ; contraintes reprises de notre échange.

TIMER1 • HORLOGE

D’où viennent les ticks ?

L’horloge donne le rythme au microcontrôleur. À 16 MHz, il y a 16 000 000 cycles par seconde. Timer1 peut recevoir une horloge issue de cette horloge, divisée par son prescaler.



Un tick du timer dure 256 / 16 000 000 = 16 µs.

F_CPU décrit l’horloge ; il ne la règle pas

Définir **F_CPU** donne une information aux outils et aux fonctions comme _delay_ms(). Cela ne change pas l’oscillateur, les fusibles ou le prescaler système. Il faut vérifier la fréquence réellement utilisée par la carte.

Grandeur	Valeur dans nos calculs
Horloge effective	16 000 000 Hz
Prescaler Timer1 N	256
Fréquence du compteur	16 000 000 / 256 = 62 500 Hz
Durée d’un tick	1 / 62 500 = 16 µs

Tous les nombres 31249, 62499 et 6249 reposent sur cette horloge effective. Si elle change, refais les calculs ; déclarer F_CPU = 16000000UL ne suffit pas à la rendre vraie.

Où vérifier : Guide fourni p. 14 et 17 ; datasheet §8, §8.11 et §16.

TIMER1 • COMPTEUR ET PRESCALER

Pourquoi un timer 16 bits ?

L’ATmega328P possède Timer0 et Timer2 sur 8 bits, et Timer1 sur 16 bits. Timer1 peut compter de **0 à 65535**, soit 65536 valeurs. TCNT1 contient sa valeur courante.

Sans division, le compteur est trop rapide

```
65536 / 16000000 = 0,004096 s
Soit environ 4,1 ms avant débordement en mode Normal.
```

Avec un prescaler de 256, 62500 ticks tiennent dans un compteur 16 bits : une période de 1 s devient facile à programmer.

CS12	CS11	CS10	Source / division
0	0	0	Timer arrêté
0	0	1	Horloge / 1
0	1	0	Horloge / 8
0	1	1	Horloge / 64
1	0	0	Horloge / 256
1	0	1	Horloge / 1024
1	1	0	Horloge externe sur T1, front descendant
1	1	1	Horloge externe sur T1, front montant

Les bits **CS12:0** sont dans TCCR1B. Ne confonds pas le prescaler Timer1 avec un éventuel prescaler de l’horloge système.

Où vérifier : Guide fourni p. 17 ; datasheet §15 et tableau 15-6, p. 110.

TIMER1 • REGISTRES

Trois réglages à distinguer

Élément	Question à laquelle il répond
Prescaler	À quelle vitesse TCNT1 compte-t-il ?
TOP	À quelle valeur le comptage recommence-t-il ?
OCR1A	Quand la sortie réagit-elle à une comparaison ?

Le rôle précis d'OCR1A dépend du mode. En CTC mode 4, OCR1A sert de TOP. En Fast PWM mode 14, ICR1 sert de TOP et OCR1A règle la durée du niveau HIGH.

TCCR1A et TCCR1B : les commandes

Registre	Bits utilisés dans ces exercices
TCCR1A	COM1A1:0 : action de la sortie OC1A WGM11:0 : partie basse du mode
TCCR1B	WGM13:2 : partie haute du mode CS12:0 : horloge et prescaler
TCNT1	Valeur du compteur sur 16 bits
OCR1A / ICR1	Valeurs sur 16 bits : comparaison et TOP selon le mode

WGM : sélectionner un mode

WGM signifie **Waveform Generation Mode**. Les quatre bits WGM13, WGM12, WGM11 et WGM10 forment une combinaison à vérifier dans le tableau des modes. Ils sont répartis dans deux registres.

Mode	WGM13:0	Fonction	TOP
4	0100	CTC	OCR1A
14	1110	Fast PWM	ICR1
15	1111	Fast PWM	OCR1A

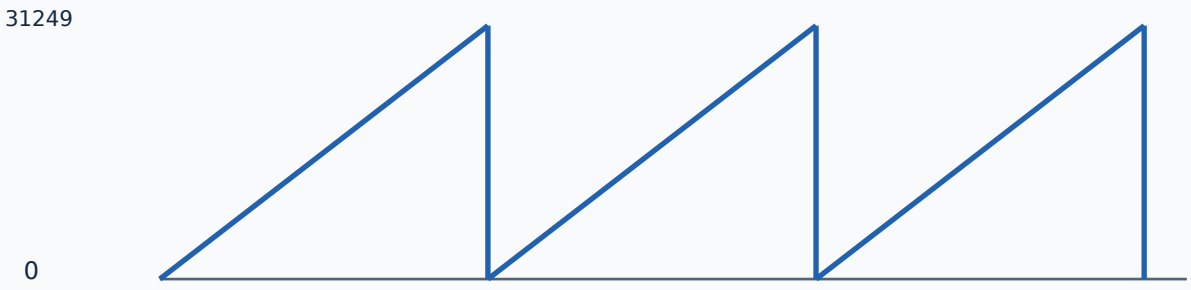
Où vérifier : Datasheet §15.11, p. 108–111 ; tableau 15-5, p. 109.

EX01 • CTC

Le timer bascule la sortie tout seul

CTC signifie Clear Timer on Compare Match. En mode 4, le compteur parcourt les valeurs de 0 à OCR1A, puis repart à 0. La comparaison peut faire basculer OC1A automatiquement.

TCNT1 : la valeur monte, puis repart à zéro



31250 ticks par séquence : 0, 1, ..., 31249.

PB1 possède la fonction alternative OC1A

OC1A signifie **Output Compare 1 A**. Une fois la fonction configurée et PB1 placée en sortie, Timer1 commande cette broche. Le CPU n’a pas à écrire dans PORTB pour chaque changement d’état.

COM1A1	COM1A0	Action en mode non PWM
0	0	Sortie OC1A déconnectée du timer
0	1	Toggle : inverse OC1A à chaque comparaison
1	0	Clear : met OC1A à LOW
1	1	Set : met OC1A à HIGH

Pour ex01, on choisit **COM1A1 = 0** et **COM1A0 = 1** : une comparaison provoque une bascule. La même combinaison COM peut avoir un autre sens dans un mode PWM.

Où vérifier : Datasheet §13.3, §15.9.2 et tableau 15-2, p. 108.

EX01 • CALCUL

Une période, deux comparaisons

On veut un signal de sortie à 1 Hz : une période complète dure 1 s. Avec toggle, il faut deux compare matches par période : un pour LOW → HIGH, un autre pour HIGH → LOW.

CTC + toggle : 0,5 s HIGH, 0,5 s LOW




```
f_timer = 16000000 / 256 = 62500 Hz
Ticks par bascule = 62500 x 0,5 = 31250
OCR1A = 31250 - 1 = 31249
```

La formule à savoir refaire

```
f_sortie = F_CPU / [2 x N x (OCR1A + 1)]
OCR1A = F_CPU / (2 x N x f_sortie) - 1
```

Le **+1** vient du comptage depuis zéro. Le **facteur 2** vient des deux bascules nécessaires à une période complète. Ce sont deux raisons différentes.

Avec F_CPU = 16 MHz, N = 256 et OCR1A = 31249 : la fréquence des comparaisons vaut 2 Hz, tandis que la fréquence du signal de sortie vaut 1 Hz.

Où vérifier : Datasheet §15.9.2 ; guide fourni p. 17.

EX01 • EXEMPLE COMMENTÉ

Configurer, puis laisser faire

```
#include <avr/io.h>

int main(void)
{
    // PB1 est une sortie ; sa fonction OC1A sera utilisée.
    DDRB |= (1 << PB1);

    // Arrêter Timer1 pendant la configuration.
    TCCR1B = 0;
    TCCR1A = 0;
    TCNT1 = 0;

    // 16 MHz / 256 = 62500 ticks/s.
    // Une bascule toutes les 31250 graduations = 0,5 s.
    OCR1A = 31249;

    // COM1A1:0 = 01 : toggle OC1A en mode non PWM.
    // WGM11:0 = 00 : partie basse du mode 4.
    TCCR1A = (1 << COM1A0);

    // WGM13:2 = 01 : mode 4 CTC, TOP = OCR1A.
    // CS12:0 = 100 : démarrer avec prescaler 256.
    TCCR1B = (1 << WGM12) | (1 << CS12);

    while (1)
    {
        // Timer1 et OC1A fonctionnent sans le CPU.
    }
}
```

La boucle reste vide. Le programme n’utilise pas PORTB pour produire le signal et n’a besoin ni d’une attente logicielle ni d’une interruption pour le clignotement.

Les bits non mentionnés valent 0 grâce à l’affectation complète avec =. Cette configuration suppose un démarrage de programme dans un environnement AVR sans autre code utilisant Timer1.

Exemple pédagogique : la validation sur ta carte reste à faire. La fréquence obtenue suit la précision de son horloge.

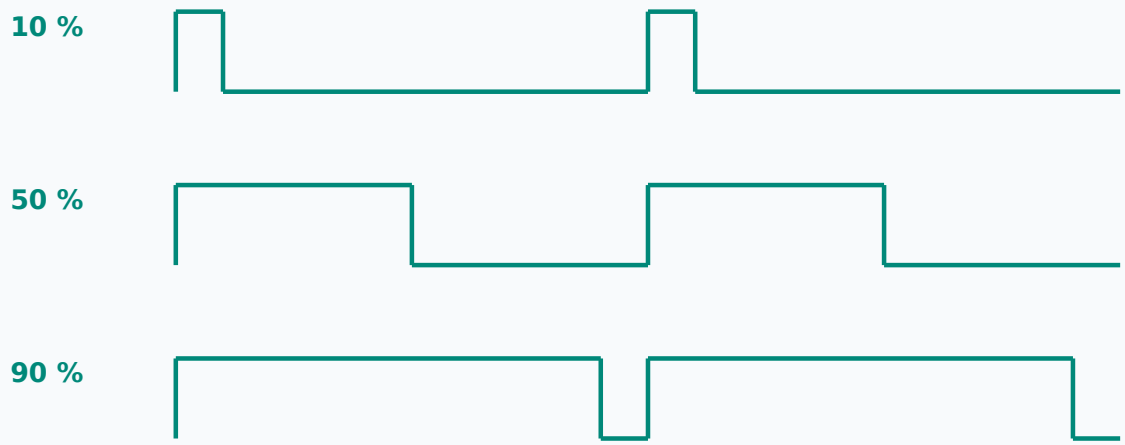
EX02 • PWM

Changer la durée allumée

Le **rapport cyclique** est la fraction de la période pendant laquelle le signal est actif. Pour une LED active à HIGH : duty = temps HIGH / période totale.

```
Fréquence : 1 Hz  -> période : 1 s
Rapport cyclique : 10 %
ON : 0,1 s        OFF : 0,9 s
```

Même période, rapports cycliques différents



Chaque motif dure 1 seconde : la fréquence reste à 1 Hz.

La fréquence décrit combien de périodes se répètent par seconde. Le rapport cyclique décrit la proportion du temps actif dans une période. On peut modifier l'un sans modifier l'autre.

Pourquoi quitter le toggle CTC ?

Un toggle à intervalles fixes donne autant de temps HIGH que LOW : 50 %. Pour obtenir 10 % automatiquement avec une boucle vide, la Fast PWM offre le comportement recherché.

À 1 Hz, la LED clignote visiblement. Pour une variation de luminosité perçue comme continue, on utiliserait généralement une PWM plus rapide ; ici, on respecte la fréquence de l'exercice.

Où vérifier : Guide fourni p. 18 ; datasheet §15.9.3.

EX02 • MODE 14

ICR1 et OCR1A ont deux rôles

Le mode **14** est Fast PWM avec TOP = ICR1. Le compteur monte de 0 à ICR1 puis repart. OCR1A reste disponible pour fixer la comparaison sur la sortie OC1A.



La fréquence reste fixe quand seul OCR1A change.

Réglage	Valeur	Signification
WGM13:0	1110	Mode 14 ; Fast PWM ; TOP = ICR1
COM1A1:0	10	OC1A non inversée
CS12:0	100	Prescaler 256

Fast PWM non inversée

La sortie est mise à HIGH au début de la période, puis à LOW lors de la comparaison. À la période suivante, le matériel recommence. Avec une LED active à HIGH, la portion HIGH correspond au temps allumé.

Pourquoi éviter le mode 15 pour cet objectif ?

En mode 15, OCR1A définit TOP. Il ne peut plus servir librement à choisir un rapport cyclique indépendant sur OC1A. Le mode 14 sépare la période dans ICR1 et la durée active dans OCR1A.

La datasheet 7810D contient une inversion de formulation dans le texte du Fast PWM. Le tableau des bits et la clarification Microchip confirment : COM1A1:0 = 10 produit la PWM non inversée.

Où vérifier : Tableaux 15-3 et 15-5 ; clarification Microchip Fast PWM, lien p. 24.

EX02 • CALCUL

1 Hz et 10 % pas à pas

1. Calculer TOP pour une période de 1 s

```
f_PWM = F_CPU / [N x (1 + ICR1)]
1 + ICR1 = 16000000 / (256 x 1) = 62500
ICR1 = 62499
```

Compter de 0 à 62499 donne 62500 ticks. Chaque tick dure 16 µs : 62500 × 16 µs = 1 s.

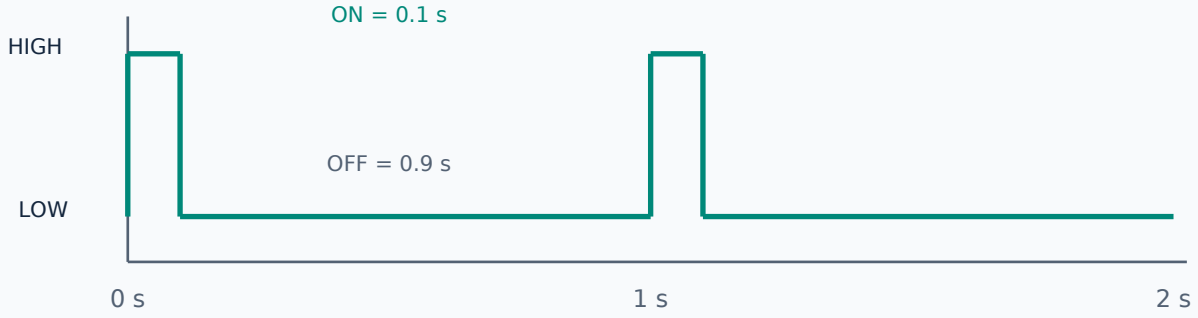
2. Calculer la durée active

```
Ticks HIGH = 62500 x 10 / 100 = 6250
OCR1A = 6250 - 1 = 6249
```

Pour les valeurs intermédiaires de cette Fast PWM non inversée, le nombre de ticks actifs correspond à OCR1A + 1. La règle utilisée ici est donc :

```
OCR1A = ((TOP + 1) x duty / 100) - 1
```

Ex02 : impulsion de 0,1 s toutes les secondes



Cas extrême à connaître
OCR1A = TOP donne une sortie constamment HIGH en mode non inversé : 100 %. OCR1A = 0 produit une impulsion très étroite ; il ne correspond pas à un vrai 0 %. L'exercice utilise 10 % à 100 %.

Où vérifier : Datasheet §15.9.3 et tableau 15-3 ; guide fourni p. 18.

EX02 • EXEMPLE COMMENTÉ

Une PWM autonome

```
#include <avr/io.h>

int main(void)
{
    // PB1 en sortie pour rendre le signal OC1A visible.
    DDRB |= (1 << PB1);

    // Timer arrêté et mode connu avant de charger les valeurs.
    TCCR1B = 0;
    TCCR1A = 0;
    TCNT1 = 0;

    // TOP fixe : 62500 ticks à 62500 Hz -> 1 seconde.
    ICR1 = 62499;

    // 6250 ticks HIGH -> 10 % de la période.
    OCR1A = 6249;

    // COM1A1:0 = 10 : PWM non inversée sur OC1A.
    // WGM11:0 = 10 : partie basse du mode 14.
    TCCR1A = (1 << COM1A1) | (1 << WGM11);

    // WGM13:2 = 11 : complète le mode 14.
    // CS12:0 = 100 : démarre Timer1 avec division par 256.
    TCCR1B = (1 << WGM13) | (1 << WGM12) | (1 << CS12);

    while (1)
    {
        // Le matériel génère la PWM en continu.
    }
}
```

Registres utilisés : DDRB, TCCR1A, TCCR1B, TCNT1, ICR1 et OCR1A. Les changements HIGH / LOW viennent du timer et de sa sortie OC1A.

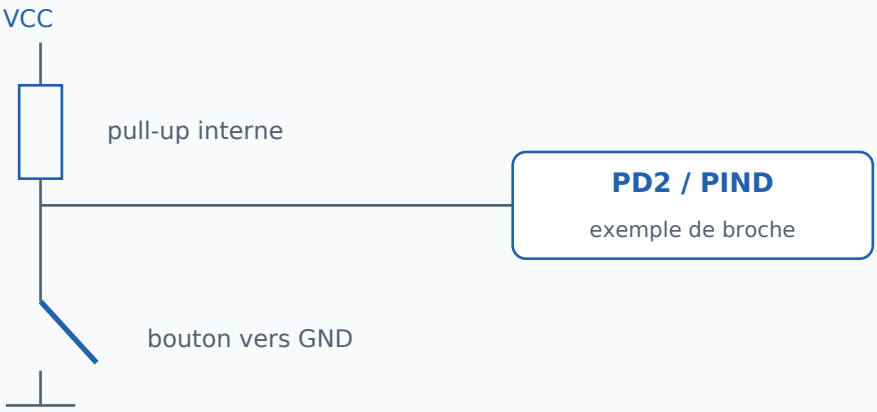
L’ordre est volontaire : arrêter le timer, charger les valeurs, puis démarrer. Dans ces exercices, ICR1 reste fixe ; seule la comparaison OCR1A changera en ex03.

EX03 • LECTURE DES BOUTONS

Appuyé peut vouloir dire zéro

Un bouton avec pull-up interne relie l’entrée à GND lorsqu’il est appuyé. La résistance maintient l’entrée à HIGH lorsqu’il est relâché. Elle évite une entrée flottante.

Entrée avec résistance de pull-up



Relâché : 1 • **Appuyé : 0**

```
// PD2 est ici un exemple, pas le mapping de SW1.
DDRD &= ~(1 << PD2); // entrée
PORTD |= (1 << PD2); // pull-up interne

if (!(PIND & (1 << PD2)))
{
    // Bouton appuyé : le bit lu vaut 0.
}
```

Le masque sélectionne le bit de PD2. Le test ! inverse le résultat logique : il est vrai quand ce bit vaut zéro. PINx sert à lire ; PORTx sert ici à configurer la pull-up.

SW1 et SW2 restent à identifier
Ne copie pas PD2 ou une autre broche au hasard. Relève, dans le schéma de ta carte, le port, le numéro du bit et la polarité de chaque bouton. Le sujet peut aussi prévoir des résistances externes.

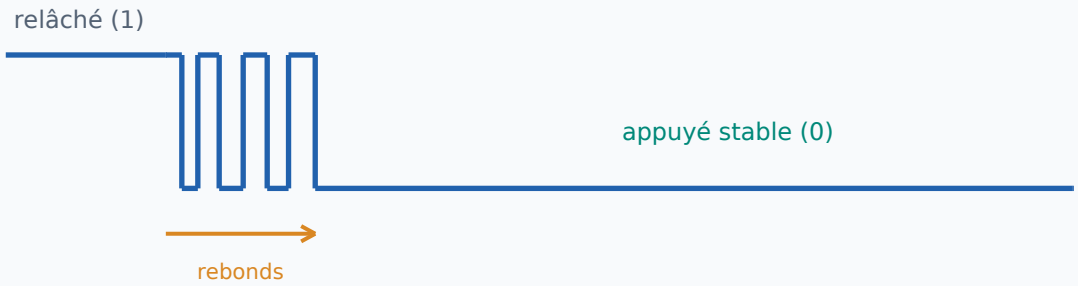
Où vérifier : Guide fourni p. 12 ; datasheet §13.2.1 et §13.2.4.

EX03 • ANTI-REBOND

Un appui doit compter une seule fois

Les contacts d’un bouton mécanique peuvent alterner rapidement entre ouvert et fermé avant de se stabiliser. Le programme pourrait interpréter ces rebonds comme plusieurs appuis.

Un appui physique peut produire plusieurs transitions



Une logique simple pour débuter

1. Détecter un passage vers l’état appuyé.
2. Attendre brièvement la stabilisation des contacts.
3. Relire et confirmer que le bouton est toujours appuyé.
4. Modifier duty une seule fois.
5. Confirmer un relâchement stable avant de réarmer.

Une temporisation de 20 ms est un exemple pédagogique de debounce, à adapter au bouton. Pour un relâchement, confirme également la stabilité : réarmer sur un seul échantillon peut laisser passer un rebond.

```
#include <util/delay.h>

_delay_ms(20); // uniquement pour le debounce
```

Dans les règles reprises de l’échange, les délais sont autorisés en ex03 seulement pour les boutons. Ils ne servent pas à produire la période de la LED : Timer1 continue à le faire.

Une attente bloquante jusqu’au relâchement est simple, mais empêche de traiter l’autre bouton pendant ce temps. Une détection d’état avec mémorisation permet de surveiller les deux.

EX03 • RAPPORT CYCLIQUE

Passer de 10 % à 100 %

Garde la configuration PWM de l’ex02 : fréquence 1 Hz, ICR1 = 62499, OC1A non inversée. Ajoute une variable duty, initialisée à 10. SW1 ajoute 10 points ; SW2 retire 10 points.

```
uint8_t duty = 10;

if (duty < 100)
    duty += 10;

// Pour le bouton de diminution :
if (duty > 10)
    duty -= 10;
```

Duty	OCR1A	Durée HIGH
10 %	6249	0,1 s
20 %	12499	0,2 s
30 %	18749	0,3 s
40 %	24999	0,4 s
50 %	31249	0,5 s
60 %	37499	0,6 s
70 %	43749	0,7 s
80 %	49999	0,8 s
90 %	56249	0,9 s
100 %	62499	HIGH constant

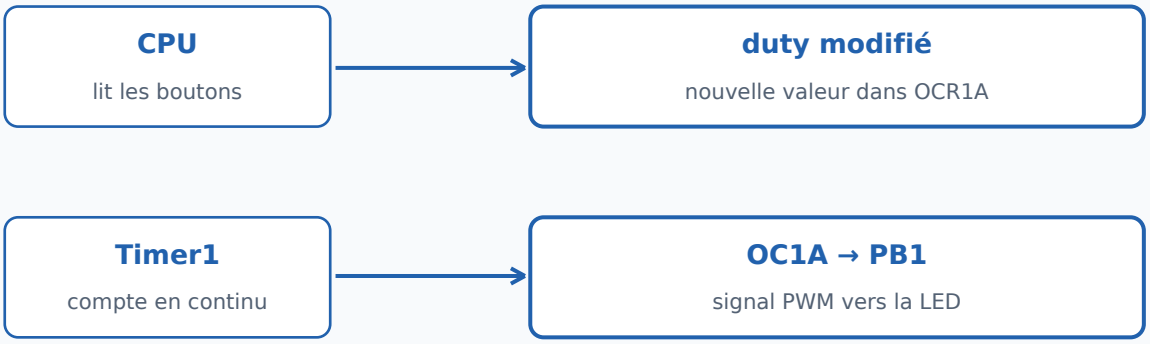
Utiliser un calcul intermédiaire sur 32 bits

```
OCR1A = (uint16_t)(
    (((uint32_t)ICR1 + 1UL) * duty) / 100UL - 1UL
);
```

Sur AVR, int est généralement sur 16 bits. Le produit 62500 × duty dépasse cette capacité. Le cast placé **avant** la multiplication impose un calcul sur 32 bits. Inclure <stdint.h> pour ces types.

EX03 • ARCHITECTURE

Le CPU lit ; le timer continue



Le debounce occupe le CPU ; le timer continue.

Le CPU décide du nouveau rapport cyclique et écrit dans OCR1A. Le matériel produit les transitions de la LED. Une attente de debounce n’interrompt pas Timer1.

Algorithme à adapter au câblage de la carte

```
Configurer PB1 et Timer1 comme en ex02
Configurer SW1 et SW2 selon le schéma de la carte
duty = 10

Répéter sans fin :
    Si nouvel appui stable de SW1 :
        Si duty < 100 : duty += 10
        Écrire la nouvelle comparaison dans OCR1A

    Si nouvel appui stable de SW2 :
        Si duty > 10 : duty -= 10
        Écrire la nouvelle comparaison dans OCR1A

    Réarmer chaque bouton après relâchement stable
```

L’appui doit être un **événement**, pas seulement un niveau maintenu. Sans mémorisation ou attente du relâchement, la boucle pourrait ajouter 10 à chaque tour et atteindre 100 % presque immédiatement.

OCR1A est double-buffered en Fast PWM : la nouvelle comparaison prend effet à la frontière d’une période. À 1 Hz, une modification peut donc attendre presque une seconde avant d’être visible. ICR1 reste fixe dans cet exercice.

Si les deux boutons sont appuyés ensemble, définis une règle cohérente avec le sujet, par exemple ignorer l’appui simultané. Le pseudocode ci-dessus ne fixe pas cette politique.

MÉTHODE • LIRE ET COMMENTER

Du comportement aux bits

Lire un registre dans le bon ordre

Pars du comportement demandé, identifie le périphérique, puis ouvre sa section Register Description. Repère la position des bits et suis les renvois vers les tableaux. Pour Timer1, lis TCCR1A et TCCR1B ensemble.

```
TCCR1A = (1 << COM1A1) | (1 << WGM11);
```

Cette ligne pose COM1A1 = 1, COM1A0 = 0, WGM11 = 1 et WGM10 = 0. Elle sélectionne la sortie non inversée **si le mode complet est Fast PWM**. TCCR1B doit compléter WGM13 et WGM12 pour obtenir 1110.

```
TCCR1B = (1 << WGM13) | (1 << WGM12) | (1 << CS12);
```

WGM13:2 = 11 complète le mode 14. CS12:0 = 100 sélectionne la division par 256. Les autres bits valent zéro, dont le bit réservé.

Un commentaire utile explique la fonction et la valeur

Évite seulement « configure timer ». Explique le mode choisi, le comportement de la sortie, le prescaler et le calcul de la période. Pour ICR1, mentionne les 62500 ticks ; pour OCR1A, mentionne la durée HIGH.

Une boucle infinie garde le firmware actif

```
while (1)
{
    // vide en ex01/ex02 ; boutons en ex03
}
```

Selon le sujet repris dans l’échange, main ne doit pas return. Sur un microcontrôleur, le comportement après la fin de main dépend du runtime ; il faut conserver la boucle infinie attendue.

Où vérifier : Guide fourni p. 24 ; datasheet §15.11.

RÉVISION • FICHE PRATIQUE

Ce que tu dois pouvoir expliquer

Question	Réponse à retenir
Pourquoi DDRB ?	PB1 doit être en sortie pour que GPIO ou OC1A puisse piloter la broche.
Pourquoi 31249 en ex01 ?	62500 ticks/s ; 31250 ticks par demi-période ; compteur à partir de zéro.
Pourquoi le facteur 2 en CTC ?	Deux bascules forment une période complète.
Pourquoi ICR1 = 62499 ?	62500 comptes à 62500 Hz donnent une période de 1 s.
Pourquoi OCR1A = 6249 ?	6250 ticks HIGH sur 62500 donnent 10 %.
Pourquoi la boucle est vide ?	Timer1 et OC1A produisent le signal sans intervention du CPU.
Pourquoi PINx pour le bouton ?	PINx lit le niveau logique de la broche.
Pourquoi un debounce ?	Un seul appui physique peut provoquer plusieurs transitions.

Les trois formules

```
CTC + toggle :
f = F_CPU / [2 x N x (OCR1A + 1)]

Fast PWM mode 14 :
f = F_CPU / [N x (ICR1 + 1)]

Fast PWM non inversée, duty 10 à 100 :
OCR1A = ((ICR1 + 1) x duty / 100) - 1
```

Pour vérifier ton programme : confirmer l’horloge, le câblage LED/boutons, les bits WGM/COM/CS, puis mesurer la période et la durée HIGH. En ex03, vérifier un seul changement par appui et les limites 10 % / 100 %.

RÉFÉRENCES • RETROUVER LES SECTIONS

Ta carte de lecture

Sujet	Guide fourni	Datasheet 7810D
Broches et ports	p. 5–6	§1 ; §13.3
Bits et masques	p. 7	§13.1 ; §13.4
DDRx / PORTx / PINx	p. 8–10	§13.2 ; §13.4, p. 72
LED et bouton	p. 11–12	§13.2 ; §28
Fonctions alternatives	p. 13	§13.3, p. 63
Horloge	p. 14	§8 ; §8.11
Timer1 / registres 16 bits	p. 17	§15, p. 89 ; §15.3, p. 91
CTC / Fast PWM	p. 17–18	§15.9.2–3, p. 100–102
COM1A non PWM / PWM	—	Tables 15-2 / 15-3, p. 108
Modes WGM	—	Table 15-5, p. 109
Clock Select / prescaler	p. 17	Table 15-6, p. 110 ; §16, p. 114
Méthode de recherche	p. 24	§15.11, p. 108–113

Les pages datasheet désignent la pagination imprimée de l'édition 7810D–AVR–01/15. Une autre édition peut changer les pages et les numéros de sections.

Document fourni : ATmega328P-guide-visuel-francais(1).pdf, 24 pages. Base des explications GPIO, horloge, timers et PWM.

Datasheet du fabricant :
ATmega328P — 7810D–AVR–01/15 (Microchip)

Clarification du fabricant :
Fast PWM Mode in ATmega328P Automotive (Microchip)

Schémas originaux et simplifiés réalisés pour ce cours. Les exemples C sont pédagogiques ; ils n'ont pas été exécutés sur la carte. Les broches SW1 / SW2 et le sujet intégral ne sont pas fournis ici.